

Introduction

Outline

- ❑ What Operating Systems Do
- ❑ Computer-System Organization
- ❑ Computer-System Architecture
- ❑ Operating-System Structure
- ❑ Operating-System Operations

What is an Operating System?

A program that acts as an intermediary between a user of a computer and the computer hardware

Operating system goals:

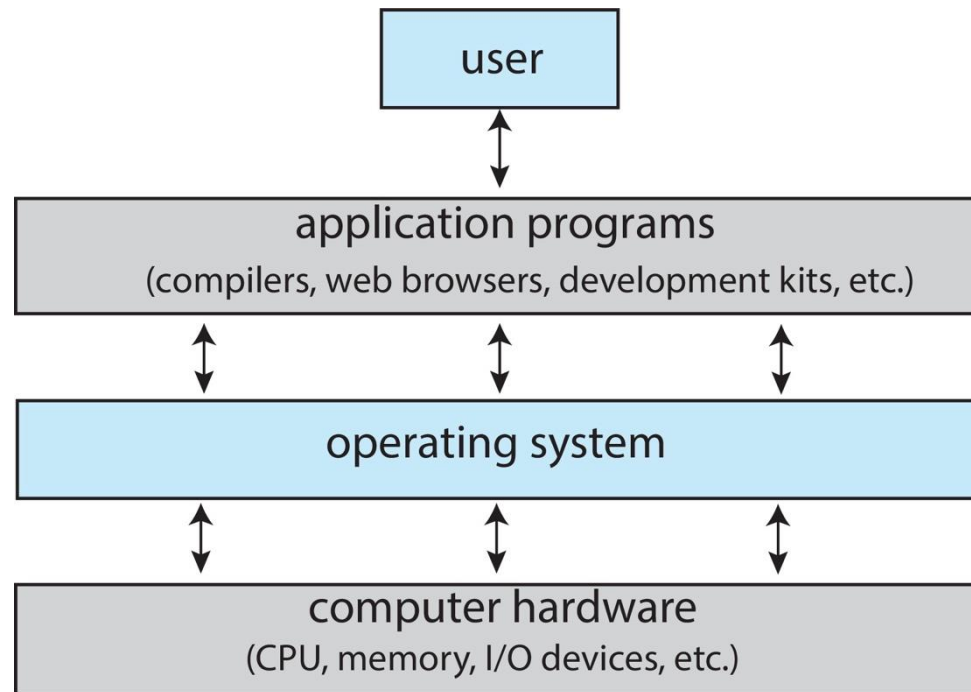
- Execute user programs and make solving user problems easier
- Make the computer system convenient to use
- Use the computer hardware in an efficient manner

Computer System Structure

Computer system can be divided into four components:

- Hardware – provides basic computing resources
 - CPU, memory, I/O devices
- Operating system
 - Controls and coordinates use of hardware among various applications and users
- Application programs – define the ways in which the system resources are used to solve the computing problems of the users
 - Word processors, compilers, web browsers, database systems, video games
- Users
 - People, machines, other computers

Abstract View of Components of Computer



What is an Operating System?...

❑ Depends on the point of view

Users want convenience, **ease of use** and **good performance**

- Don't care about **resource utilization**

But shared computer such as **mainframe** or **minicomputer** must keep all users happy

Users of dedicated systems such as **workstations** have dedicated resources but frequently use shared resources from **servers**

Handheld computers are resource poor, optimized for usability and battery life

Some computers have little or no user interface, such as embedded computers in devices and automobiles

Operating System Definition

OS is a **resource allocator**

- Manages all resources
- Decides between conflicting requests for efficient and fair resource use

OS is a **control program**

- Controls execution of programs to prevent errors and improper use of the computer

Operating System Definition...

No universally accepted definition

“Everything a vendor ships when you order an operating system” is a good approximation

- But varies wildly

“The one program running at all times on the computer” is the **kernel**.

Everything else is either

- a system program (ships with the operating system) , or
- an application program.

Computer Startup

bootstrap program is loaded at power-up or reboot

- Typically stored in ROM or EPROM, generally known as **firmware**
- Initializes all aspects of system
- Loads operating system kernel and starts execution

Computer-System Operation

I/O devices and the CPU can execute concurrently

Each device controller is in charge of a particular device type

Each device controller has a local buffer

CPU moves data from/to main memory to/from local buffers

I/O is from the device to local buffer of controller

Device controller informs CPU that it has finished its operation by causing an **interrupt**

Common Functions of Interrupts

Interrupt transfers control to the interrupt service routine generally, through the **interrupt vector**, which contains the addresses of all the service routines

Interrupt architecture must save the address of the interrupted instruction

A **trap** or **exception** is a software-generated interrupt caused either by an error or a user request

An operating system is **interrupt driven**

Interrupt Handling

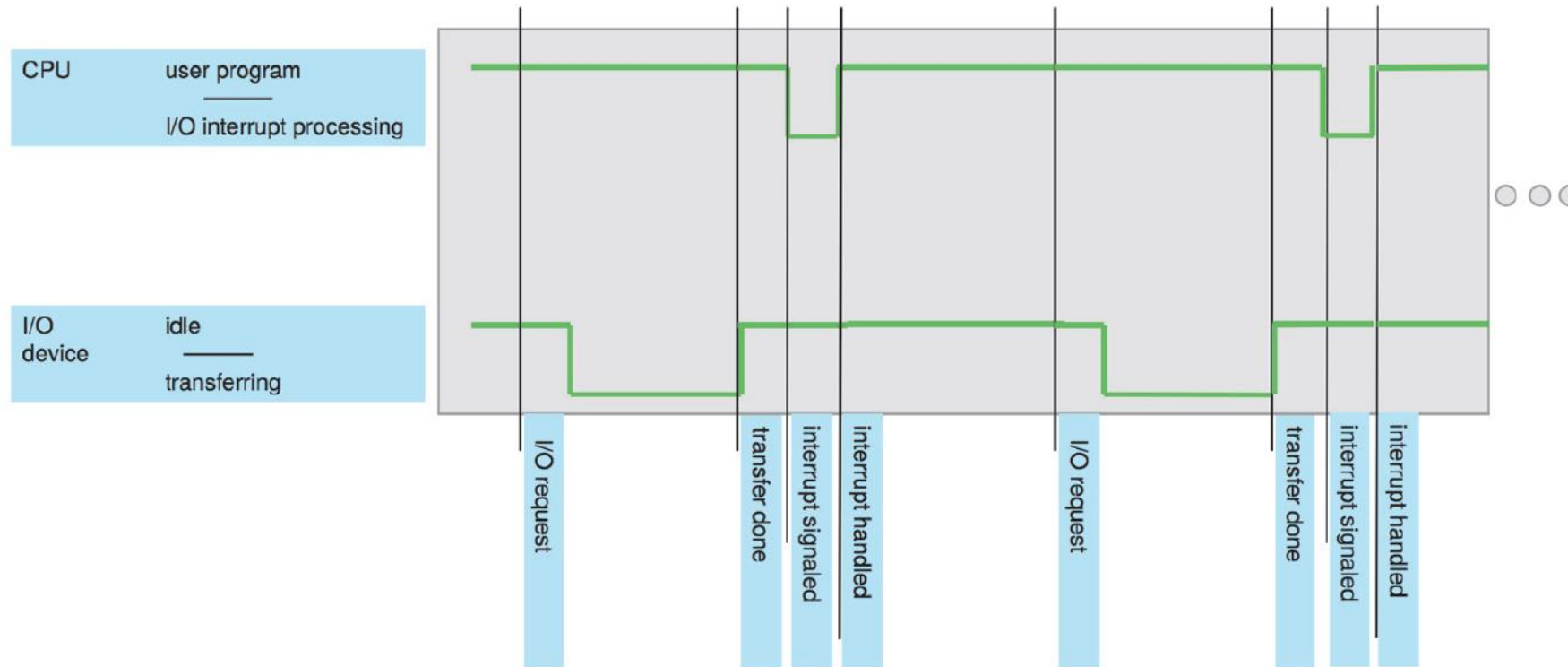
The operating system preserves the state of the CPU by storing registers and the program counter

Determines which type of interrupt has occurred:

- **polling**
- **vectored** interrupt system

Separate segments of code determine what action should be taken for each type of interrupt

Interrupt Timeline



Operating-System Operations

Interrupt driven (hardware and software)

- Hardware interrupt by one of the devices
- Software interrupt (**exception** or **trap**):
 - Software error (e.g., division by zero)
 - Request for operating system service
 - Other process problems include infinite loop, processes modifying each other or the operating system

Intel Pentium processor event-vector table.

vector number	description
0	divide error
1	debug exception
2	null interrupt
3	breakpoint
4	INTO-detected overflow
5	bound range exception
6	invalid opcode
7	device not available
8	double fault
9	coprocessor segment overrun (reserved)
10	invalid task state segment
11	segment not present
12	stack fault
13	general protection
14	page fault
15	(Intel reserved, do not use)
16	floating-point error
17	alignment check
18	machine check
19–31	(Intel reserved, do not use)
32–255	maskable interrupts

Operating-System Operations ...

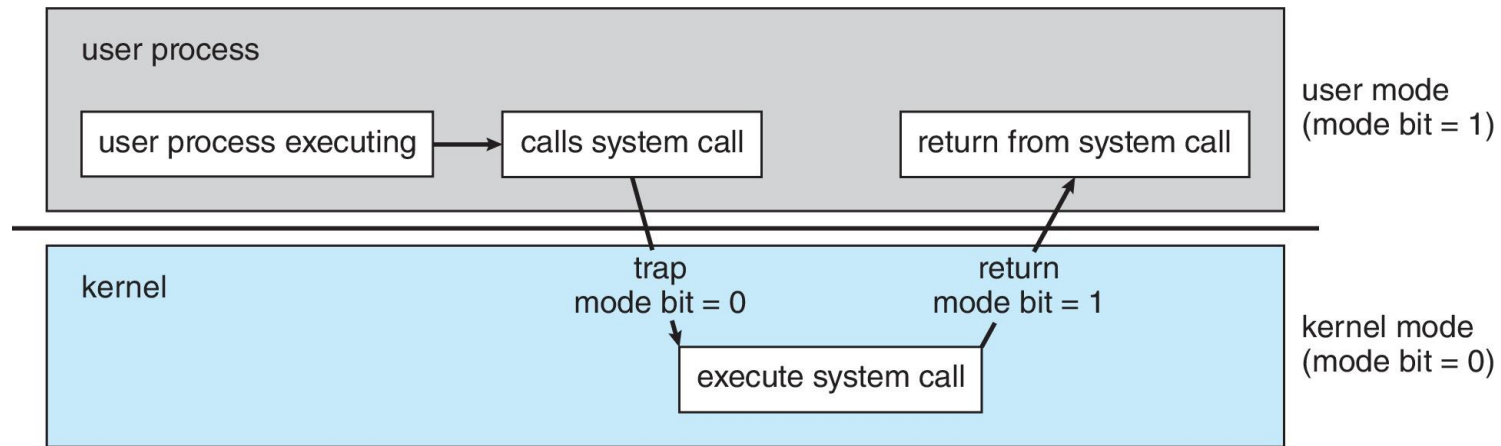
Dual-mode operation allows OS to protect itself and other system components

- **User mode** and **kernel mode**
- **Mode bit** provided by hardware
 - Provides ability to distinguish when system is running user code or kernel code
 - Some instructions designated as **privileged**,
 - only executable in kernel mode
 - System call changes mode to kernel, return from call resets it to user

Increasingly CPUs support multi-mode operations

- i.e. **virtual machine manager (VMM)** mode for guest **VMs**

Transition from User to Kernel Mode



Computer-System Architecture

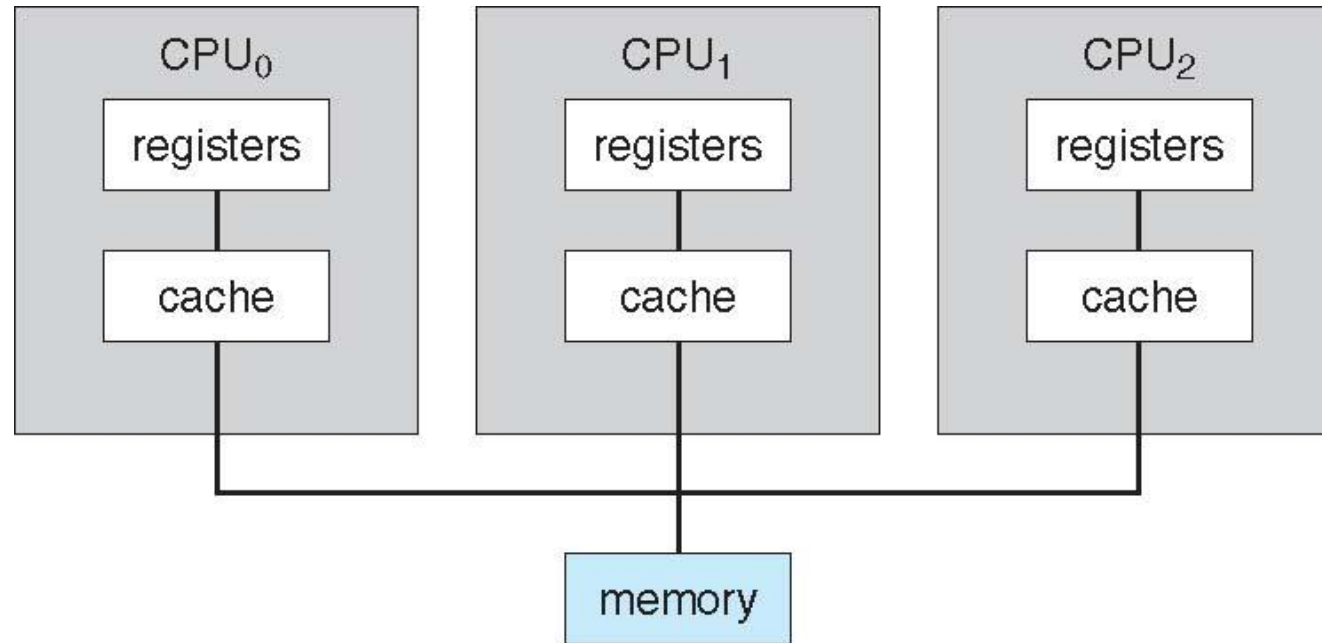
Most systems use a single general-purpose processor

- Most systems have special-purpose processors as well

Multiprocessors systems growing in use and importance

- Also known as **parallel systems**, **tightly-coupled systems**
- Advantages include:
 1. **Increased throughput**
 2. **Economy of scale**
 3. **Increased reliability** – graceful degradation or fault tolerance
- Two types:
 1. **Asymmetric Multiprocessing** – each processor is assigned a specific task.
 2. **Symmetric Multiprocessing** – each processor performs all tasks

Symmetric Multiprocessing Architecture

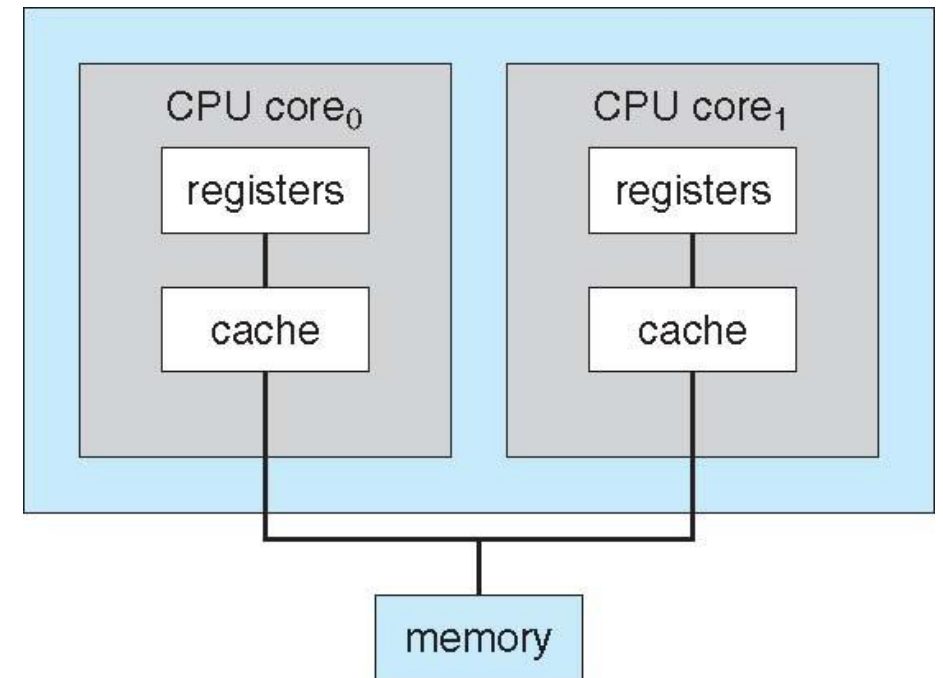


A Dual-Core Design

Multi-chip and **multicore**

Systems containing all chips

- Chassis containing multiple separate systems

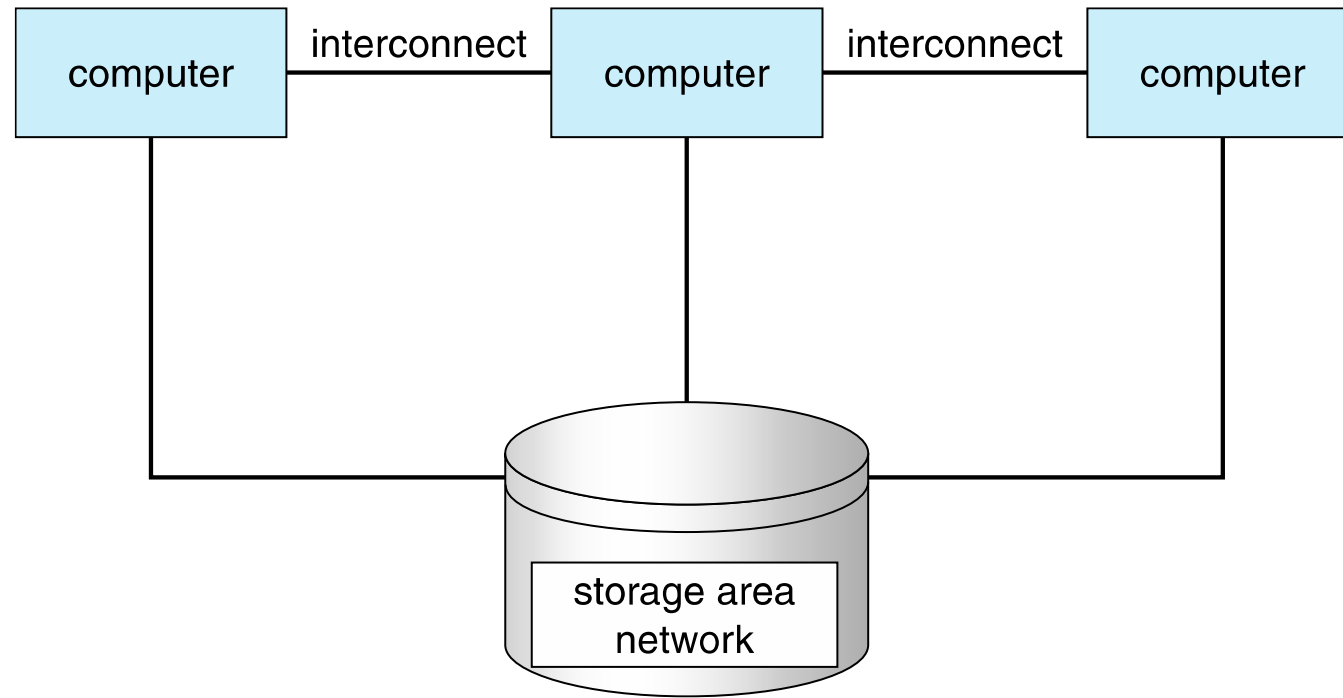


Clustered Systems

Like multiprocessor systems, but multiple systems working together

- Usually sharing storage via a **storage-area network (SAN)**
- Provides a **high-availability** service which survives failures
 - **Asymmetric clustering** has one machine in hot-standby mode
 - **Symmetric clustering** has multiple nodes running applications, monitoring each other
- Some clusters are for **high-performance computing (HPC)**
 - Applications must be written to use **parallelization**

Clustered Systems...



Operating System Structure

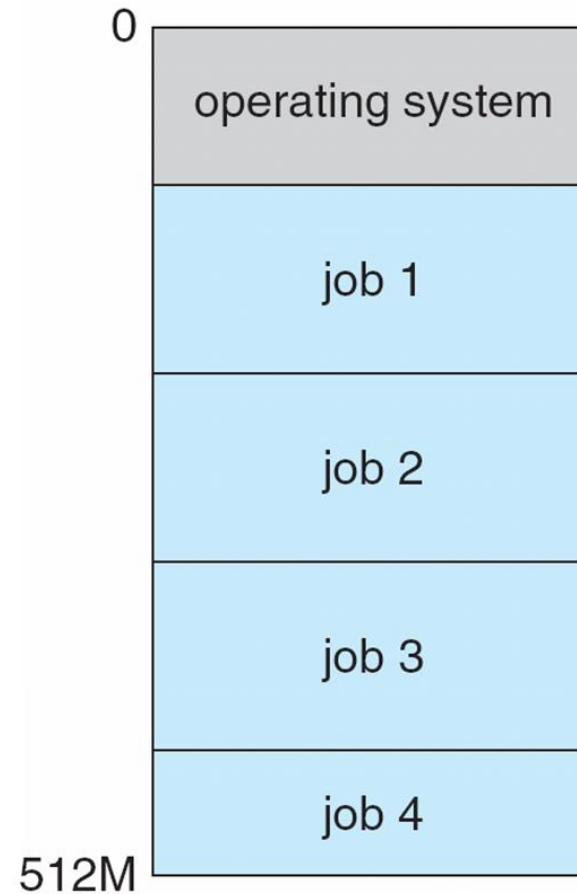
Multiprogramming (**Batch system**) needed for efficiency

- Single user cannot keep CPU and I/O devices busy at all times
- Multiprogramming organizes jobs (code and data) so CPU always has one to execute
- A subset of total jobs in system is kept in memory
- One job selected and run via **job scheduling**
- When it has to wait (for I/O for example), OS switches to another job

Timesharing (**multitasking**) is logical extension in which CPU switches jobs so frequently that users can interact with each job while it is running, creating **interactive** computing

- **Response time** should be < 1 second
- Each user has at least one program executing in memory ⇒ **process**
- If several jobs ready to run at the same time ⇒ **CPU scheduling**
- If processes don't fit in memory, **swapping** moves them in and out to run
- **Virtual memory** allows execution of processes not completely in memory

Memory Layout for Multiprogrammed System



Open-Source Operating Systems

Operating systems made available in source-code format rather than just binary **closed-source**

Counter to the **copy protection** and **Digital Rights Management (DRM)** movement

Started by **Free Software Foundation (FSF)**, which has “copyleft” **GNU Public License (GPL)**

Examples include **GNU/Linux** and **BSD UNIX** (including core of **Mac OS X**), and many more

Can use VMM like VMware Player (Free on Windows), Virtualbox (open source and free on many platforms - <http://www.virtualbox.com>)

- Use to run guest operating systems for exploration

Operating-System Structures

Outline

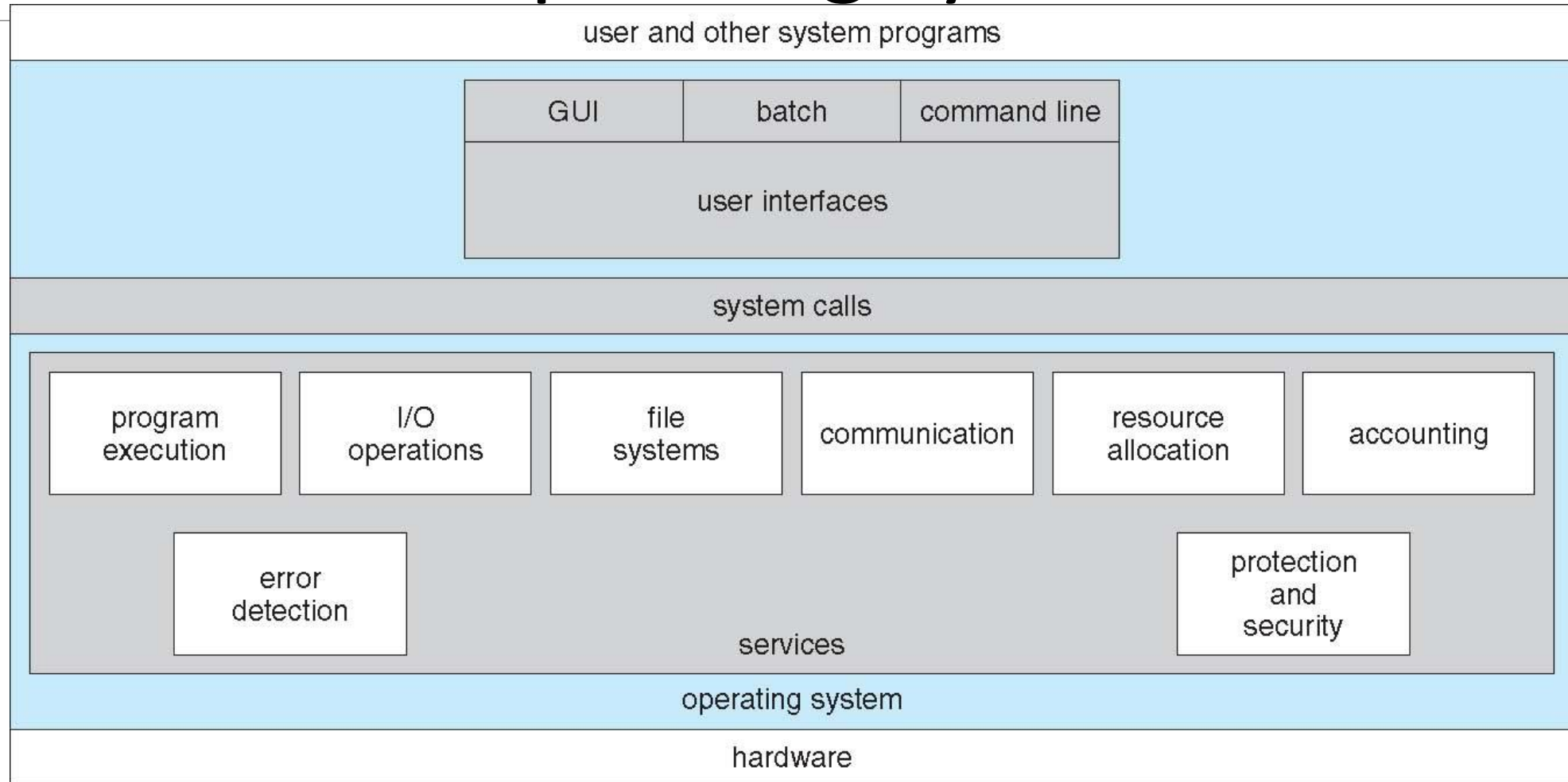
- ❑ Operating System Services
- ❑ User Operating System Interface
- ❑ System Calls
- ❑ Types of System Calls
- ❑ Operating System Structure

Operating System Structure

OS view from three main points:

1. Services that the it provides: UI, I/O operations, File-system manipulation, Communications, Error detection, etc.
2. Based on the interface that it makes available to users and programmers: CLI, GUI, Touch Screen, Batch.
3. Based on its components and their interconnections: Monolithic Structure, Layered Approach, Microkernels, Modules, Hybrid Systems.

A View of Operating System Services



1. Operating System Services

Operating systems provide an environment for execution of programs and services to programs and users

One set of operating-system services provides functions that are helpful to the user:

- **User interface** - Almost all operating systems have a user interface (**UI**).
 - Varies between **Command-Line (CLI)**, **Graphics User Interface (GUI)**, **Batch**
- **Program execution** - The system must be able to load a program into memory and to run that program, end execution, either normally or abnormally (indicating error)
- **I/O operations** - A running program may require I/O, which may involve a file or an I/O device

Operating System Services...

One set of operating-system services provides functions that are helpful to the user (Cont.):

- **File-system manipulation** - The file system is of particular interest. Programs need to read and write files and directories, create and delete them, search them, list file information, permission management.
- **Communications** – Processes may exchange information, on the same computer or between computers over a network
 - Communications may be via shared memory or through message passing (packets moved by the OS)
- **Error detection** – OS needs to be constantly aware of possible errors
 - May occur in the CPU and memory hardware, in I/O devices, in user program
 - For each type of error, OS should take the appropriate action to ensure correct and consistent computing
 - Debugging facilities can greatly enhance the user's and programmer's abilities to efficiently use the system

Operating System Services...

Another set of OS functions exists for ensuring the efficient operation of the system itself via resource sharing

- **Resource allocation** - When multiple users or multiple jobs running concurrently, resources must be allocated to each of them
 - Many types of resources - CPU cycles, main memory, file storage, I/O devices.
- **Accounting** - To keep track of which users use how much and what kinds of computer resources
- **Protection and security** - The owners of information stored in a multiuser or networked computer system may want to control use of that information, concurrent processes should not interfere with each other
 - **Protection** involves ensuring that all access to system resources is controlled
 - **Security** of the system from outsiders requires user authentication, extends to defending external I/O devices from invalid access attempts

2. User and Operating-System Interface

Ways to interface with the operating system:

- CLI
- GUI
- Touch-Screen Interface

User Operating System Interface – CLI

CLI or **command interpreter** allows direct command entry

- Sometimes implemented in kernel, sometimes by systems program
- Sometimes multiple flavors implemented – **shells**
- Primarily fetches a command from user and executes it
- Sometimes commands built-in, sometimes just names of programs
- If the latter, adding new features doesn't require shell modification

CLI...

```
1. root@r6181-d5-us01:~ (ssh)
X root@r6181-d5-u... %1 X ssh %2 X root@r6181-d5-us01... %3
Last login: Thu Jul 14 08:47:01 on ttys002
iMacPro:~ pbg$ ssh root@r6181-d5-us01
root@r6181-d5-us01's password:
Last login: Thu Jul 14 06:01:11 2016 from 172.16.16.162
[root@r6181-d5-us01 ~]# uptime
 06:57:48 up 16 days, 10:52,  3 users,  load average: 129.52, 80.33, 56.55
[root@r6181-d5-us01 ~]# df -kh
Filesystem      Size  Used Avail Use% Mounted on
/dev/mapper/vg_ks-lv_root
                  50G   19G   28G  41% /
tmpfs            127G  520K  127G   1% /dev/shm
/dev/sda1        477M   71M  381M  16% /boot
/dev/dssd0000    1.0T  480G  545G  47% /dssd_xfs
tcp://192.168.150.1:3334/orangefs
                  12T   5.7T   6.4T  47% /mnt/orangefs
/dev/gpfs-test   23T   1.1T   22T   5% /mnt/gpfs
[root@r6181-d5-us01 ~]#
[root@r6181-d5-us01 ~]# ps aux | sort -nrk 3,3 | head -n 5
root      97653 11.2  6.6 42665344 17520636 ?    S<Ll  Jul13 166:23 /usr/lpp/mmfs/bin/mmfsd
root      69849  6.6  0.0      0      0 ?        S    Jul12 181:54 [vpthread-1-1]
root      69850  6.4  0.0      0      0 ?        S    Jul12 177:42 [vpthread-1-2]
root      3829  3.0  0.0      0      0 ?        S    Jun27 730:04 [rp_thread 7:0]
root      3826  3.0  0.0      0      0 ?        S    Jun27 728:08 [rp_thread 6:0]
[root@r6181-d5-us01 ~]# ls -l /usr/lpp/mmfs/bin/mmfsd
-r-x----- 1 root root 20667161 Jun  3  2015 /usr/lpp/mmfs/bin/mmfsd
[root@r6181-d5-us01 ~]#
```

User Operating System Interface - GUI

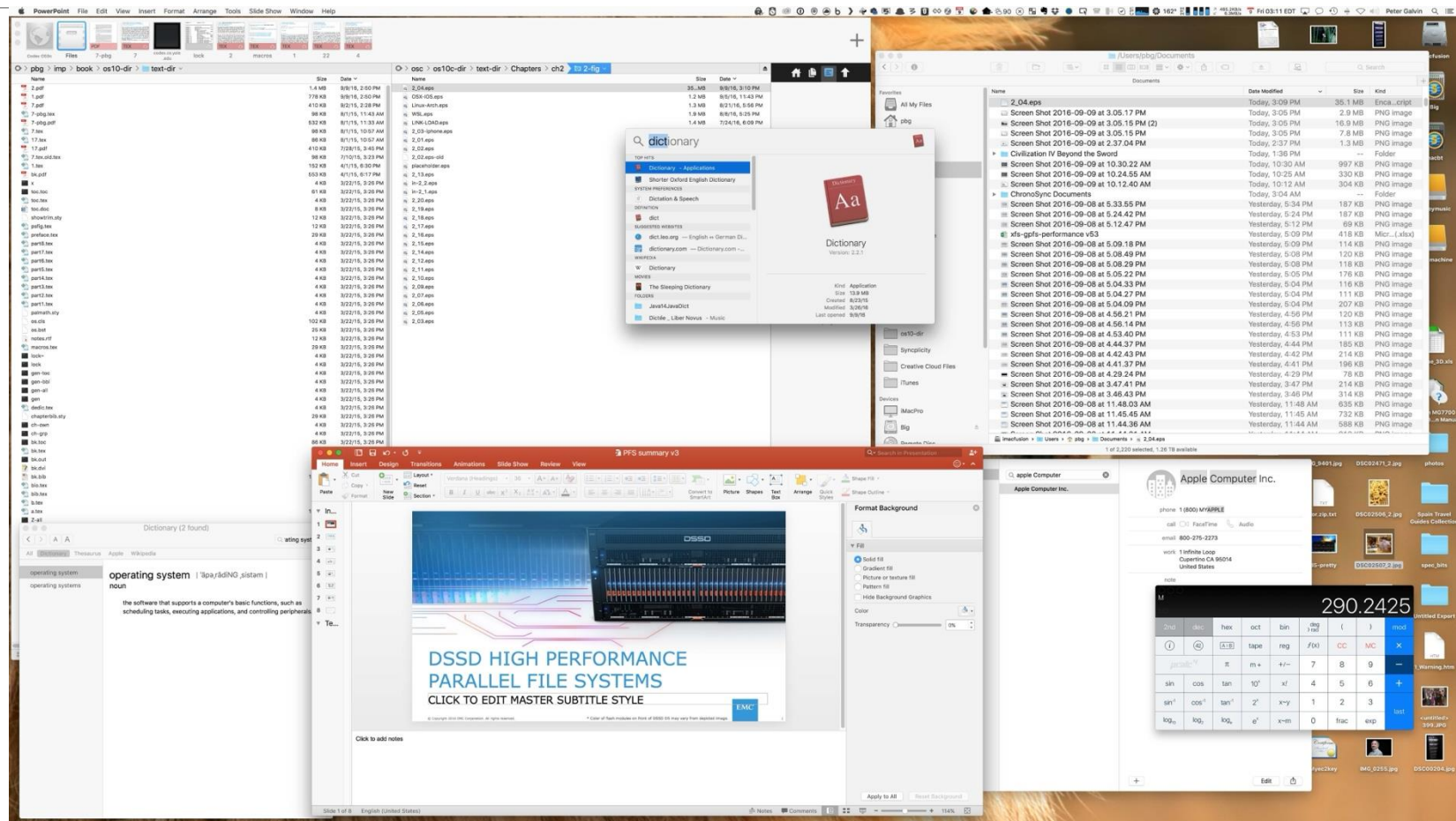
User-friendly **desktop** metaphor interface

- Usually mouse, keyboard, and monitor
- **Icons** represent files, programs, actions, etc
- Various mouse buttons over objects in the interface cause various actions (provide information, options, execute function, open directory (known as a **folder**))
- Invented at Xerox PARC

Many systems now include both CLI and GUI interfaces

- Microsoft Windows is GUI with CLI “command” shell
- Apple Mac OS X is “Aqua” GUI interface with UNIX kernel underneath and shells available
- Unix and Linux have CLI with optional GUI interfaces (CDE, KDE, GNOME)

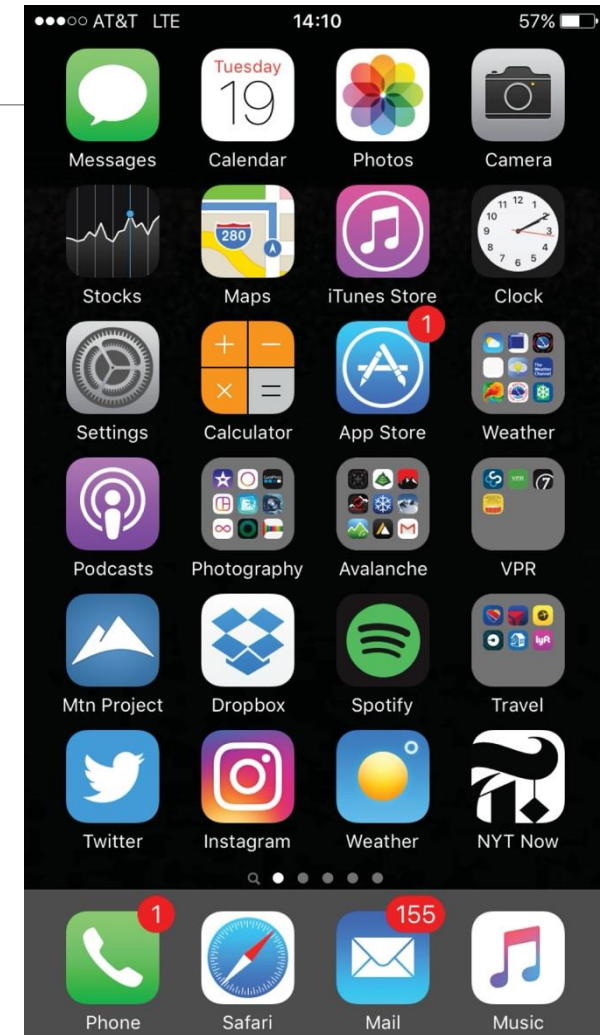
The Mac OS X GUI



Touchscreen Interfaces

Touchscreen devices require new interfaces

- | Mouse not possible or not desired
- | Actions and selection based on gestures
- | Virtual keyboard for text entry
- | Voice commands.



System Calls

Programming interface to the services provided by the OS

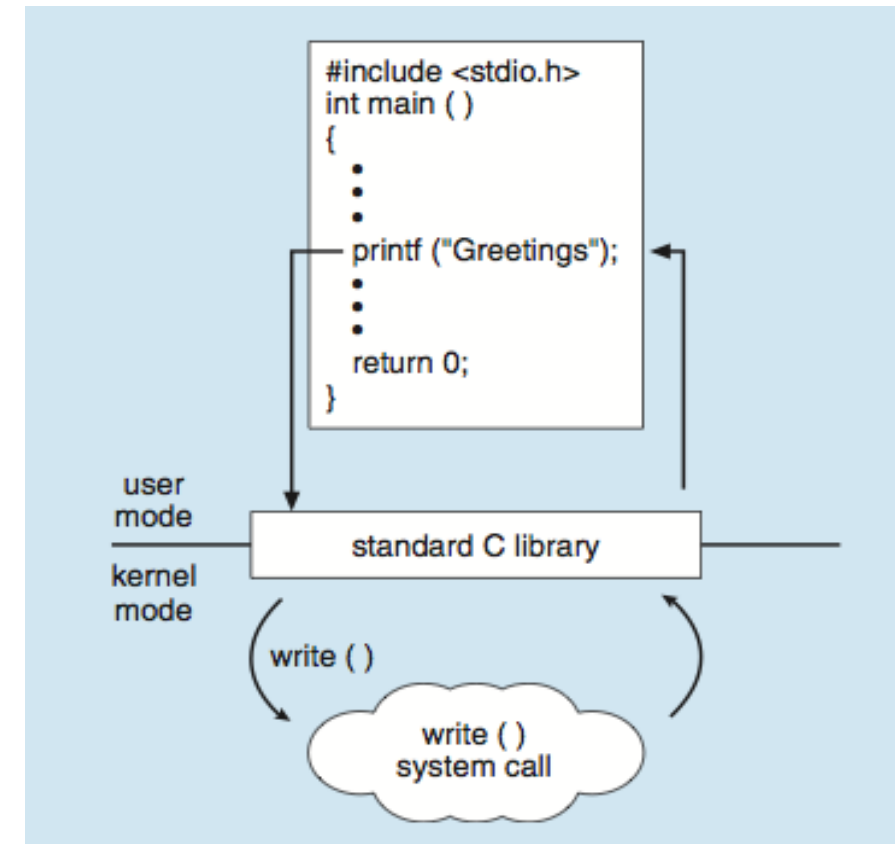
Typically written in a high-level language (C or C++)

Mostly accessed by programs via a high-level **Application Programming Interface (API)** rather than direct system call use

Three most common APIs are Win32 API for Windows, **POSIX API** for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS X), and Java API for the Java virtual machine (JVM)

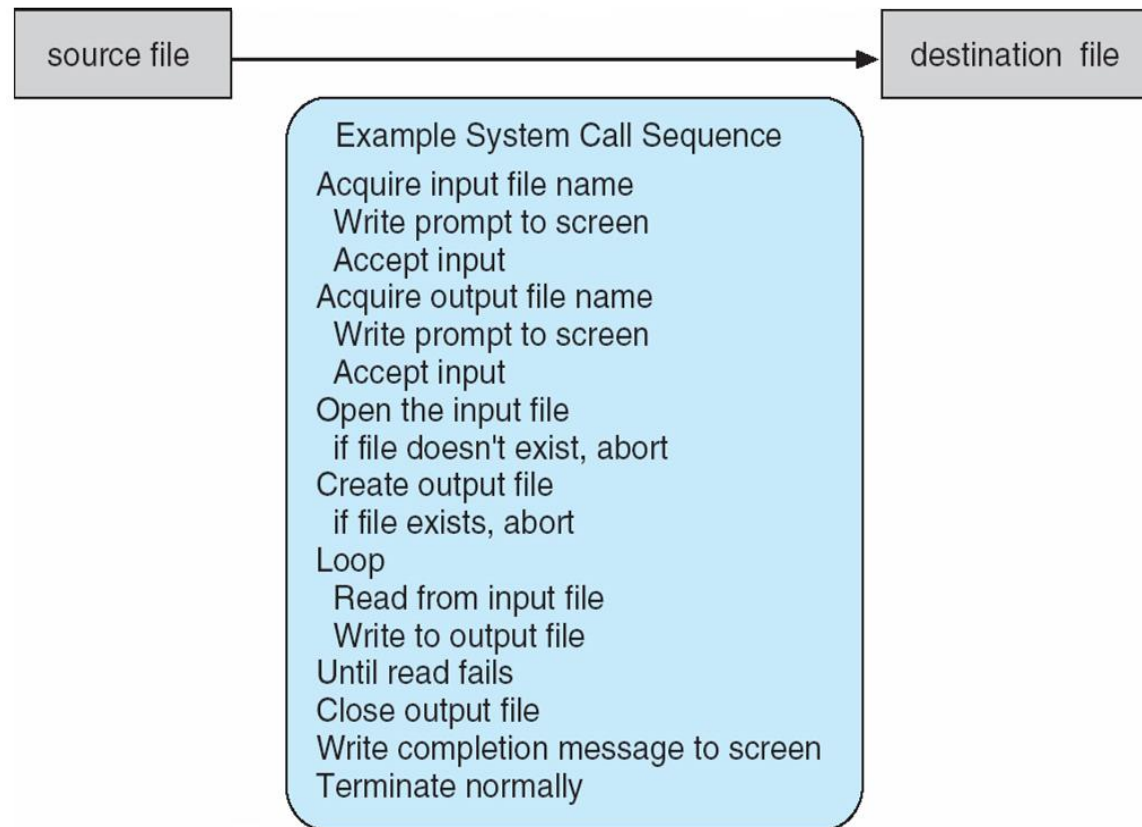
Standard C Library Example

C program invoking printf() library call, which calls write() system call



Example of System Calls

System call sequence to copy the contents of one file to another file



System Call Implementation

Typically, a number associated with each system call

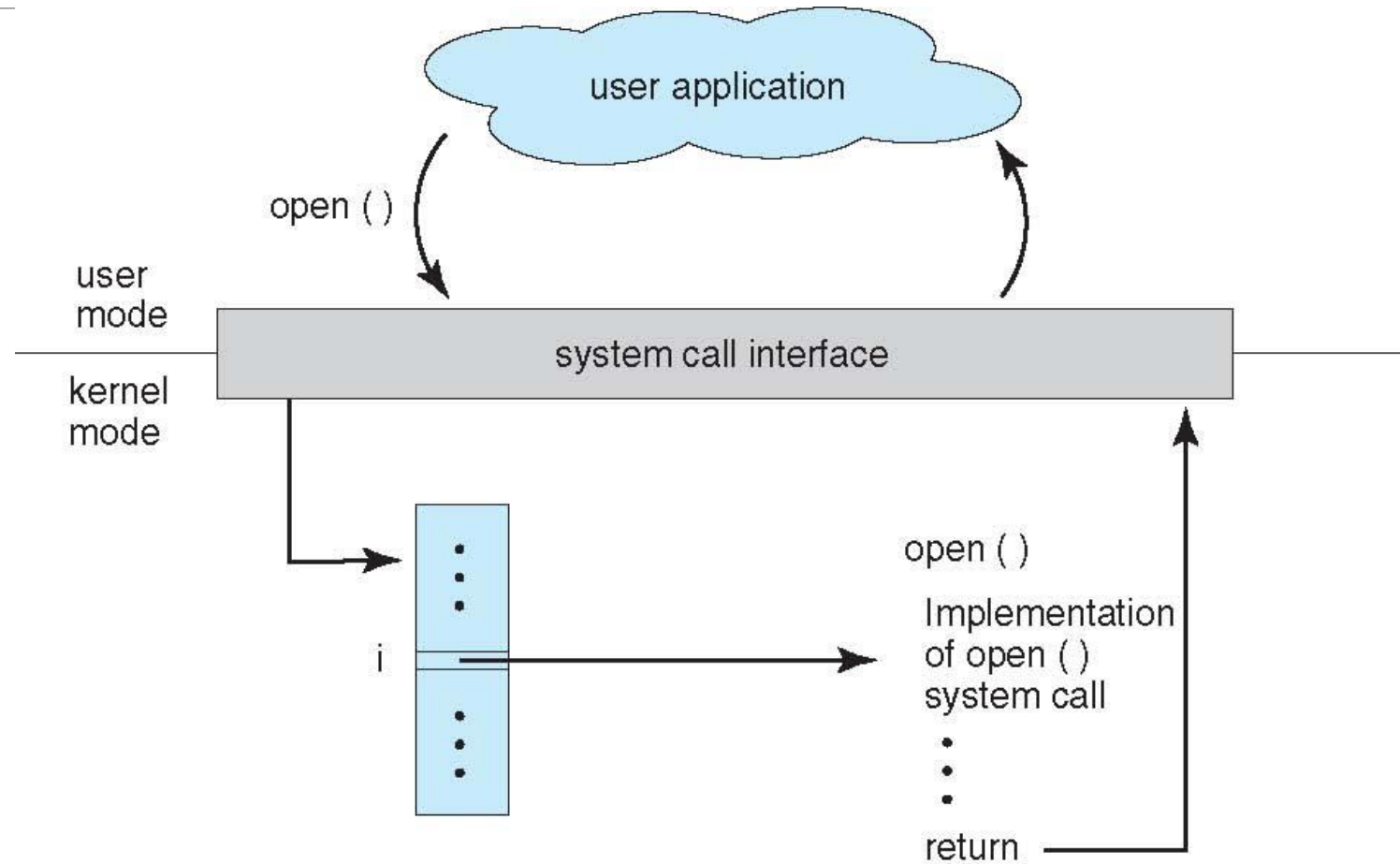
- **System-call interface** maintains a table indexed according to these numbers

The system call interface invokes the intended system call in OS kernel and returns status of the system call and any return values

The caller need know nothing about how the system call is implemented

- Just needs to obey API and understand what OS will do as a result call
- Most details of OS interface hidden from programmer by API
 - Managed by run-time support library (set of functions built into libraries included with compiler)

API – System Call – OS Relationship



System Call Parameter Passing

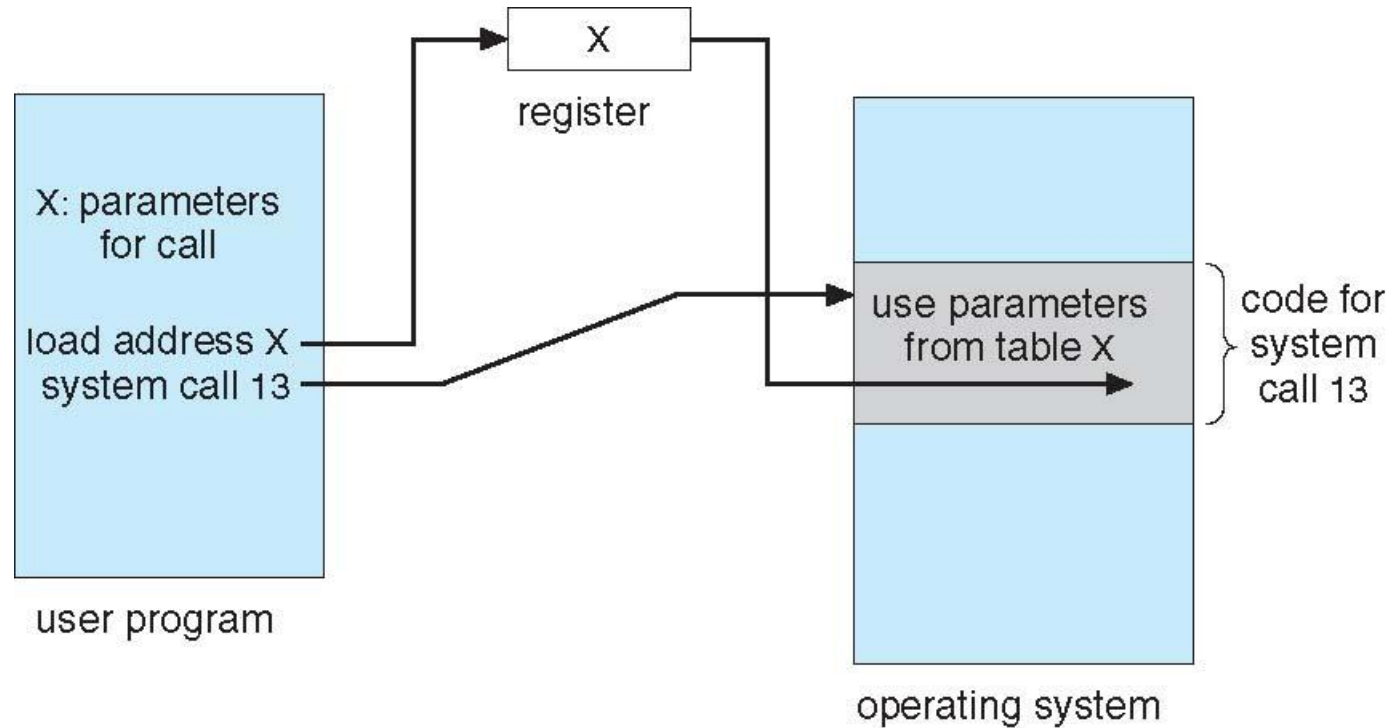
Often, more information is required than simply identity of desired system call

- Exact type and amount of information vary according to OS and call

Three general methods used to pass parameters to the OS

- Simplest: pass the parameters in registers
 - In some cases, may be more parameters than registers
- Parameters stored in a block, or table, in memory, and address of block passed as a parameter in a register
 - This approach taken by Linux and Solaris
- Parameters placed, or **pushed**, onto the **stack** by the program and **popped** off the stack by the operating system
- Block and stack methods do not limit the number or length of parameters being passed

Parameter Passing via Table



Types of System Calls

- **Process control**
- **File management**
- **Device management**
- **Information maintenance**
- **Communications**
- **Protection**

Types of System Calls

Process control

- create process, terminate process
- end, abort
- load, execute
- get process attributes, set process attributes
- wait for time
- wait event, signal event
- allocate and free memory
- Dump memory if error
- **Debugger** for determining **bugs, single step** execution
- **Locks** for managing access to shared data between processes

Types of System Calls...

File management

- create file, delete file
- open, close file
- read, write, reposition
- get and set file attributes

Device management

- request device, release device
- read, write, reposition
- get device attributes, set device attributes
- logically attach or detach devices

Types of System Calls...

Information maintenance

- get time or date, set time or date
- get system data, set system data
- get and set process, file, or device attributes

Communications

- create, delete communication connection
- send, receive messages if **message passing model** to **host name** or **process name**
 - From **client** to **server**
- **Shared-memory model** create and gain access to memory regions
- transfer status information
- attach and detach remote devices

Types of System Calls...

Protection

- Control access to resources
- Get and set permissions
- Allow and deny user access

Examples of Windows and Unix System Calls

	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shmget() mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

3. OS components and their interconnections

General-purpose OS is very large program

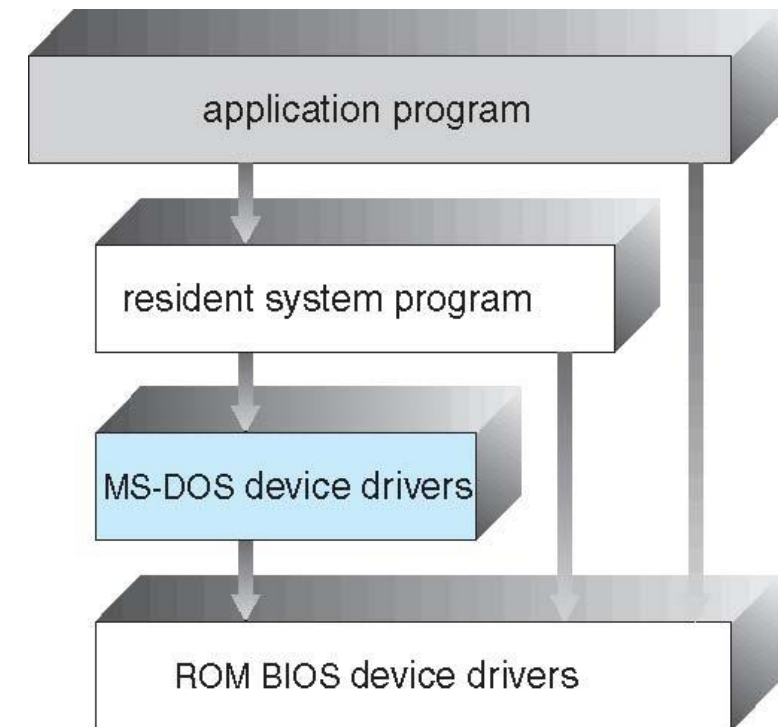
Various ways to structure ones

- Simple structure – MS-DOS
- More complex – UNIX (Monolithic Structure)
- Layered – an abstraction
- Microkernel –Mach
- Modules
- **Hybrid**

Simple Structure -- MS-DOS

MS-DOS – written to provide the most functionality in the least space

- Not divided into modules
- Although MS-DOS has some structure, its interfaces and levels of functionality are not well separated

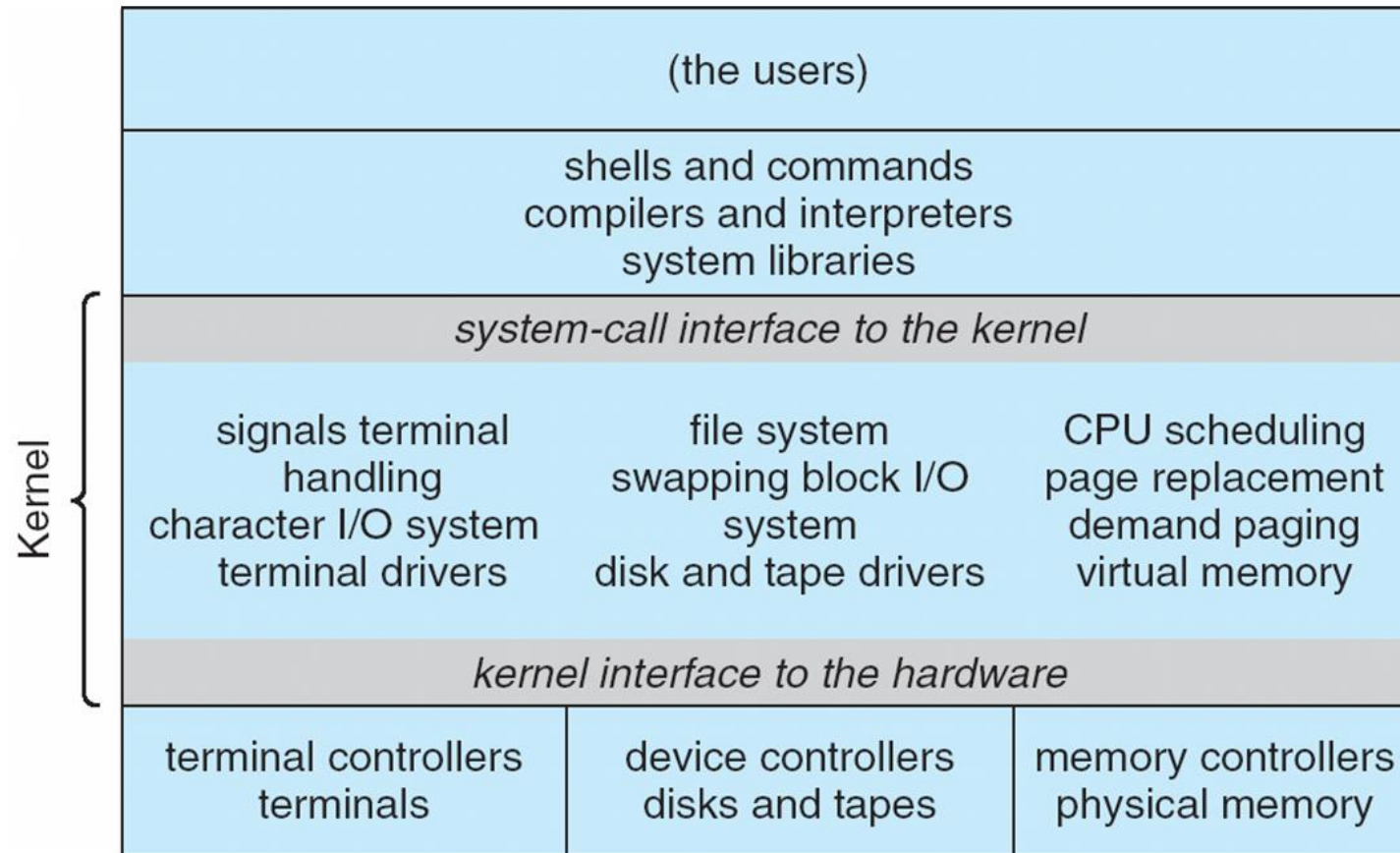


Monolithic Structure – Original UNIX

UNIX – limited by hardware functionality, the original UNIX operating system had limited structuring. The UNIX OS consists of two separable parts

- Systems programs
- The kernel
 - Consists of everything below the system-call interface and above the physical hardware
 - Provides the file system, CPU scheduling, memory management, and other operating-system functions; a large number of functions for one level

Traditional UNIX System Structure

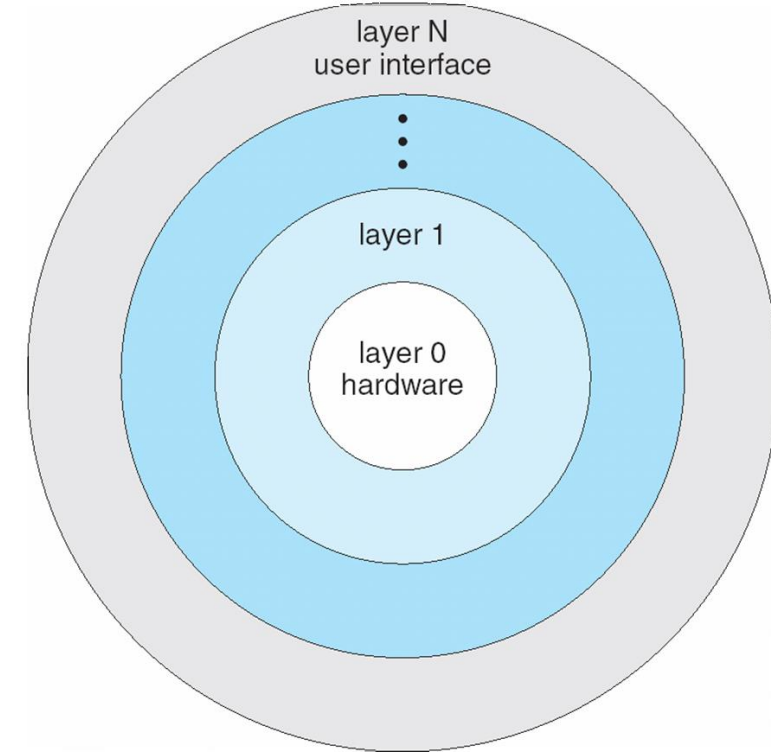


Beyond simple but not fully layered

Layered Approach

The operating system is divided into a number of layers (levels), each built on top of lower layers. The bottom layer (layer 0), is the hardware; the highest (layer N) is the user interface.

With modularity, layers are selected such that each uses functions (operations) and services of only lower-level layers



Microkernel System Structure

Moves as much from the kernel into user space

Mach example of **microkernel**

- Mac OS X kernel (**Darwin**) partly based on Mach

Communication takes place between user modules using **message passing**

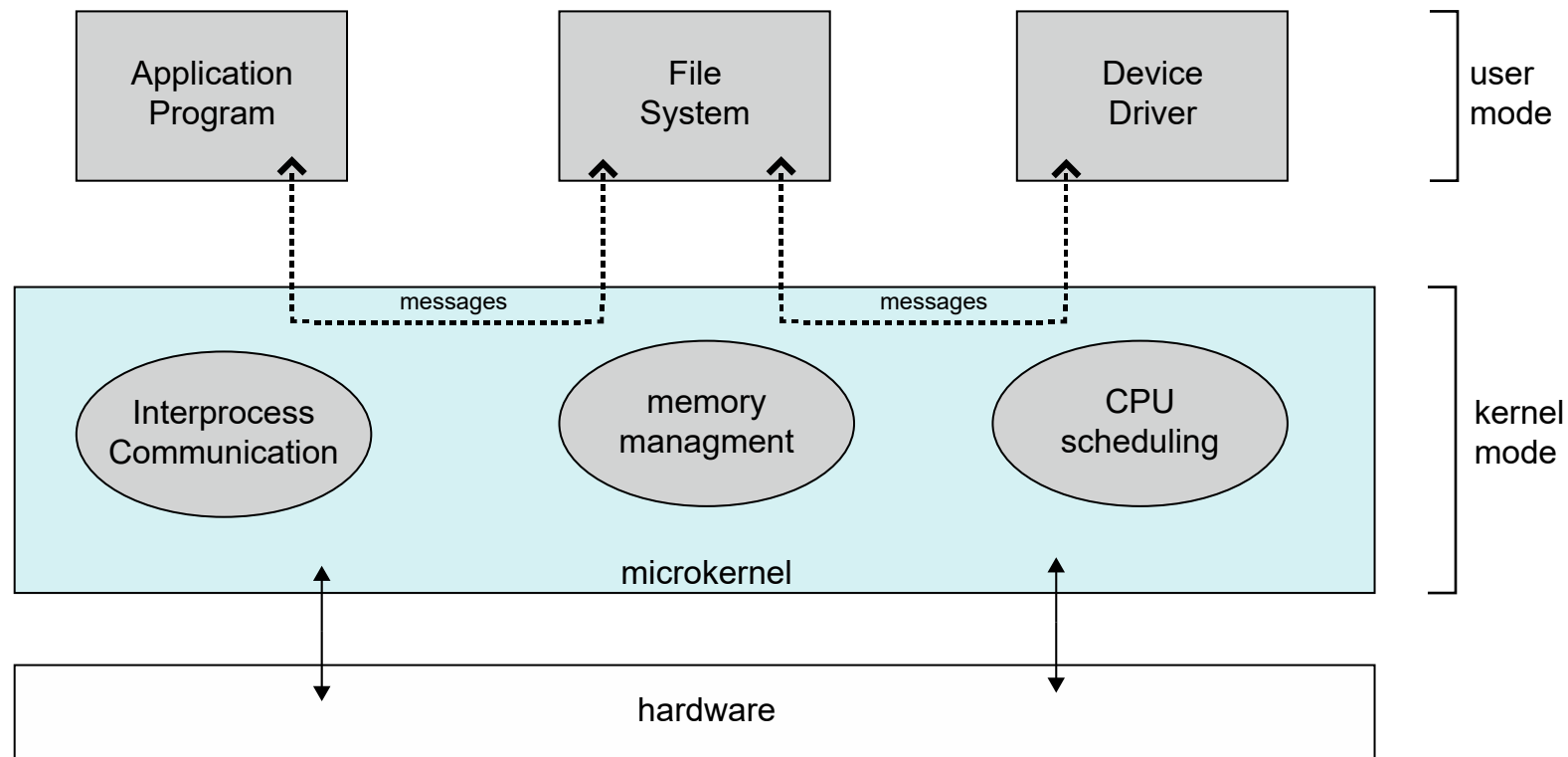
Benefits:

- Easier to extend a microkernel
- Easier to port the operating system to new architectures
- More reliable (less code is running in kernel mode)
- More secure

Detriments:

- Performance overhead of user space to kernel space communication

Microkernel System Structure ...



Modules

Many modern operating systems implement **loadable kernel modules**

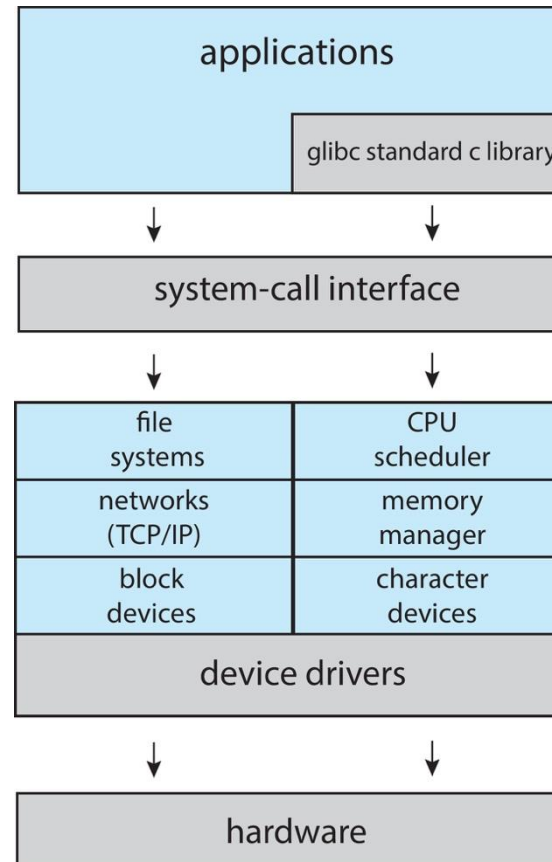
- Uses object-oriented approach
- Each core component is separate
- Each talks to the others over known interfaces
- Each is loadable as needed within the kernel

Overall, similar to layers but with more flexible

- Linux, Solaris, etc

Linux System Structure

Monolithic plus modular design



Hybrid Systems

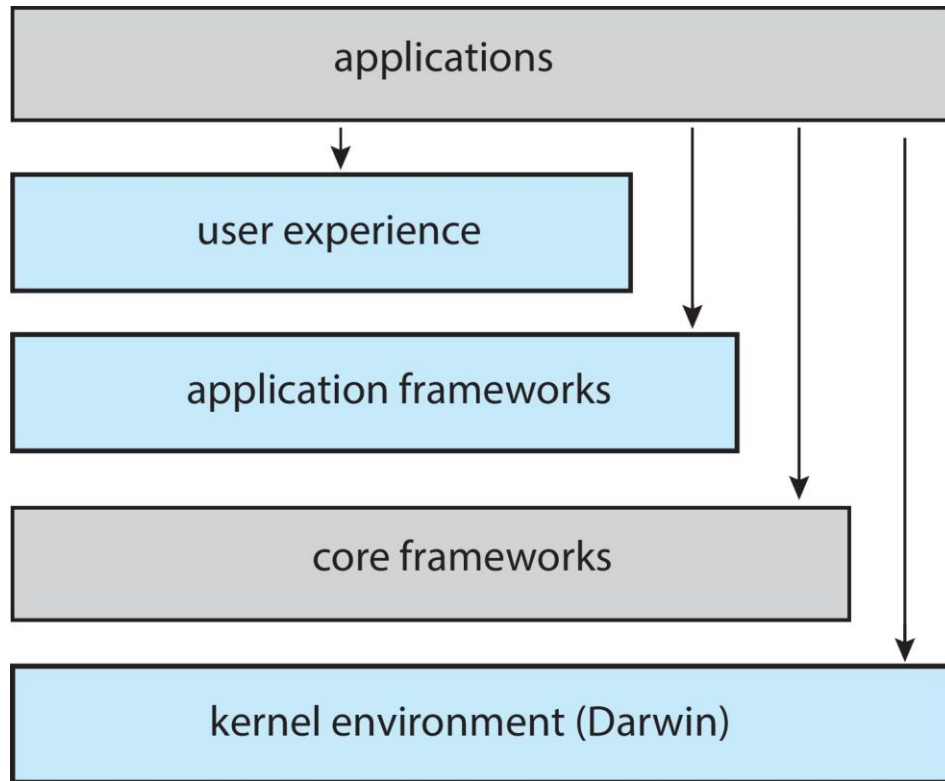
Most modern operating systems are actually not one pure model

- Hybrid combines multiple approaches to address performance, security, usability needs
- Linux and Solaris kernels in kernel address space, so monolithic, plus modular for dynamic loading of functionality
- Windows mostly monolithic, plus microkernel for different subsystem *personalities*

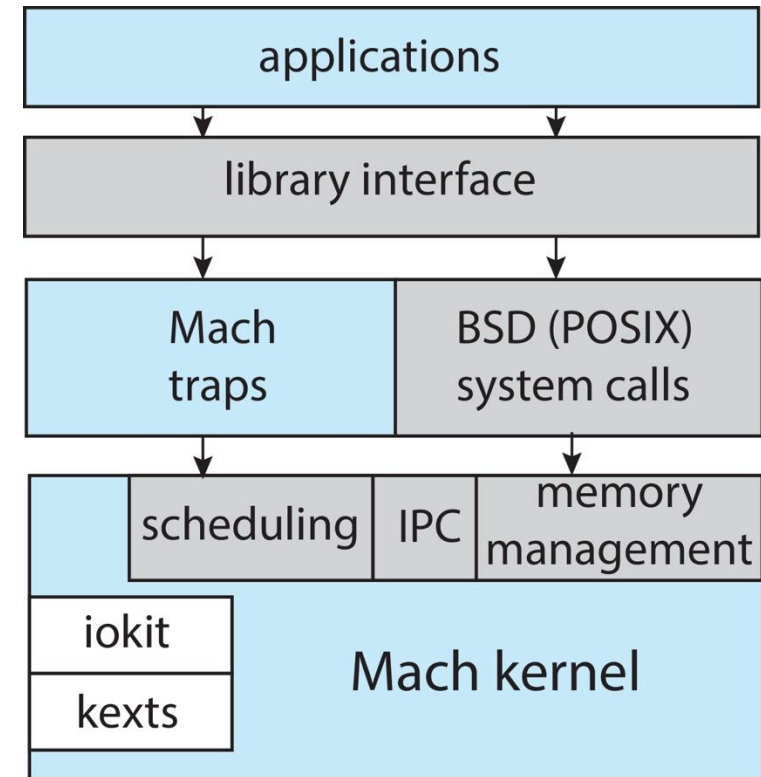
Apple Mac OS X hybrid, layered, **Aqua** UI plus **Cocoa** programming environment

- Below is kernel consisting of Mach microkernel and BSD Unix parts, plus I/O kit and dynamically loadable modules (called **kernel extensions**)

Mac OS X Structure



macOS and iOS Structure



Darwin

iOS

Apple mobile OS for *iPhone, iPad*

- Structured on Mac OS X, added functionality
- Does not run OS X applications natively
 - Also runs on different CPU architecture (ARM vs. Intel)
- **Cocoa Touch** Objective-C API for developing apps
- **Media services** layer for graphics, audio, video
- **Core services** provides cloud computing, databases
- Core operating system, based on Mac OS X kernel

Cocoa Touch

Media Services

Core Services

Core OS

Android

Developed by Open Handset Alliance (mostly Google)

- Open Source

Similar stack to IOS

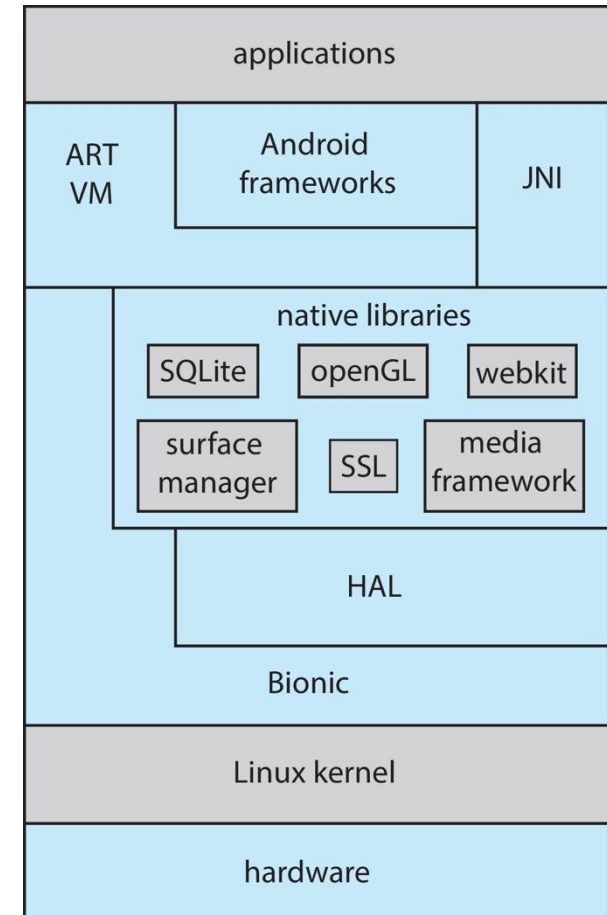
Based on Linux kernel but modified

- Provides process, memory, device-driver management
- Adds power management

Runtime environment includes core set of libraries and Dalvik virtual machine

- Apps developed in Java plus Android API
 - Java class files compiled to Java bytecode then translated to executable then runs in Dalvik VM

Libraries include frameworks for web browser (webkit), database (SQLite), multimedia, smaller libc



Operating-System Debugging

Debugging is finding and fixing errors, or **bugs**

OS generate **log files** containing error information

Failure of an application can generate **core dump** file capturing memory of the process

Operating system failure can generate **crash dump** file containing kernel memory

Beyond crashes, performance tuning can optimize system performance

- Sometimes using ***trace listings*** of activities, recorded for analysis
- **Profiling** is periodic sampling of instruction pointer to look for statistical trends

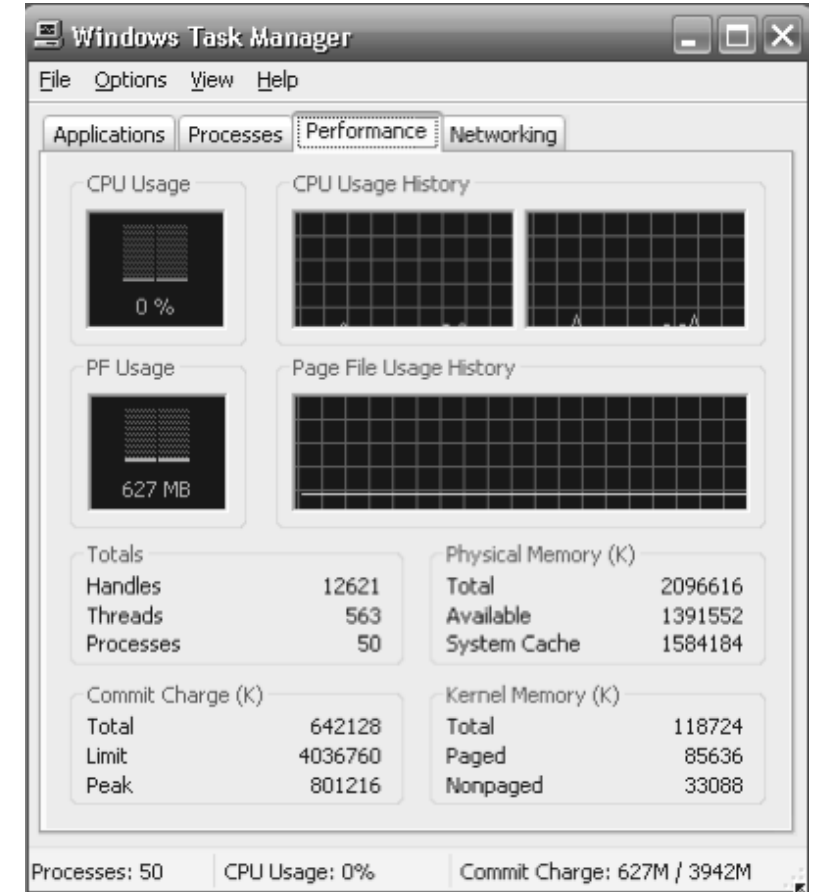
Kernighan's Law: "Debugging is twice as hard as writing the code in the first place. Therefore, if you write the code as cleverly as possible, you are, by definition, not smart enough to debug it."

Performance Tuning

Improve performance by removing bottlenecks

OS must provide means of computing and displaying measures of system behavior

For example, “top” program or Windows Task Manager



Computer System Environments

Traditional

Mobile

Client Server

Peer-to-Peer

Cloud computing

Real-time Embedded

Computing Environments - Traditional

Stand-alone general purpose machines

But blurred as most systems interconnect with others (i.e., the Internet)

Portals provide web access to internal systems

Mobile computers interconnect via **wireless networks**

Networking becoming ubiquitous – even home systems use **firewalls** to protect home computers from Internet attacks

Computing Environments - Mobile

Handheld smartphones, tablets, etc

What is the functional difference between them and a “traditional” laptop?

Extra feature – more OS features (GPS, gyroscope)

Allows new types of apps like ***augmented reality***

Use IEEE 802.11 wireless, or cellular data networks for connectivity

Leaders are **Apple iOS** and **Google Android**

Computing Environments – Distributed

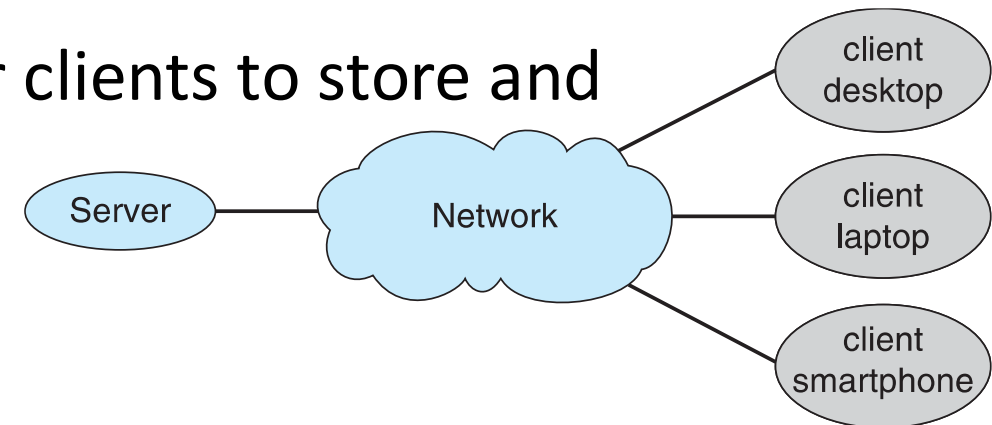
Distributed computing

- Collection of separate, possibly heterogeneous, systems networked together
- **Network** is a communications path, **TCP/IP** most common
 - **Local Area Network (LAN)**
 - **Wide Area Network (WAN)**
 - **Metropolitan Area Network (MAN)**
 - **Personal Area Network (PAN)**

Computing Environments – Client-Server

Client-Server Computing

- Dumb terminals supplanted by smart PCs
- Many systems now **servers**, responding to requests generated by **clients**
- **Compute-server system** provides an interface to client to request services (i.e., database)
- **File-server system** provides interface for clients to store and retrieve files

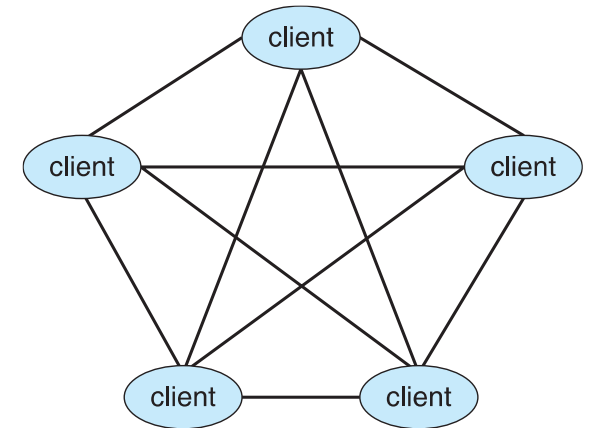


Computing Environments - Peer-to-Peer

Another model of distributed system

P2P does not distinguish clients and servers

- Instead all nodes are considered peers
- May each act as client, server or both
- Examples include Napster and Gnutella, **Voice over IP (VoIP)** such as Skype



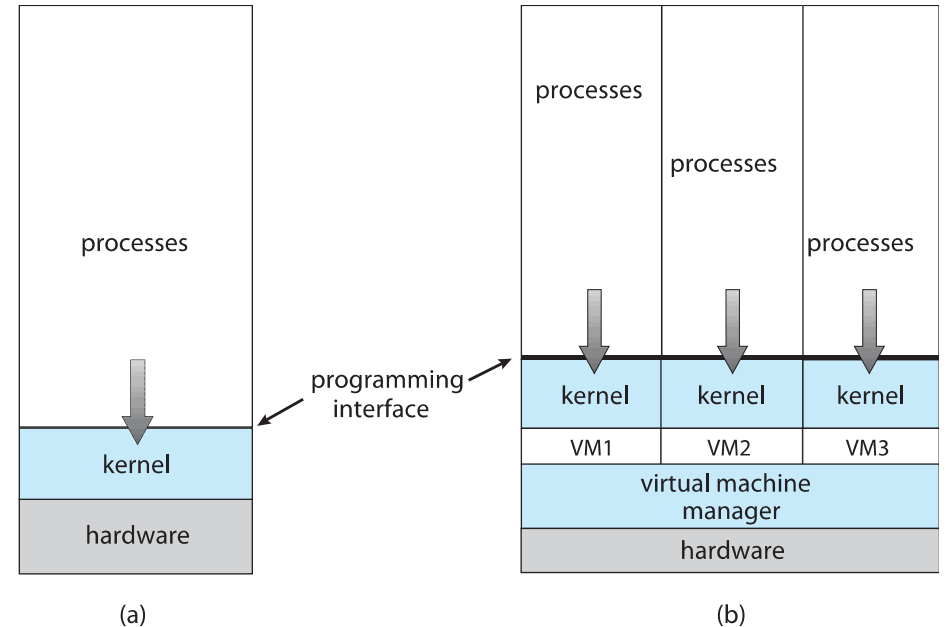
Computing Environments - Virtualization

Allows operating systems to run applications within other OSes

- Vast and growing industry

Emulation used when source CPU type different from target type (i.e. PowerPC to Intel x86)

Virtualization – OS natively compiled for CPU, running **guest** OSes also natively compiled



Computing Environments – Cloud Computing

Delivers computing, storage, even apps as a service across a network

Logical extension of virtualization because it uses virtualization as the base for its functionality.

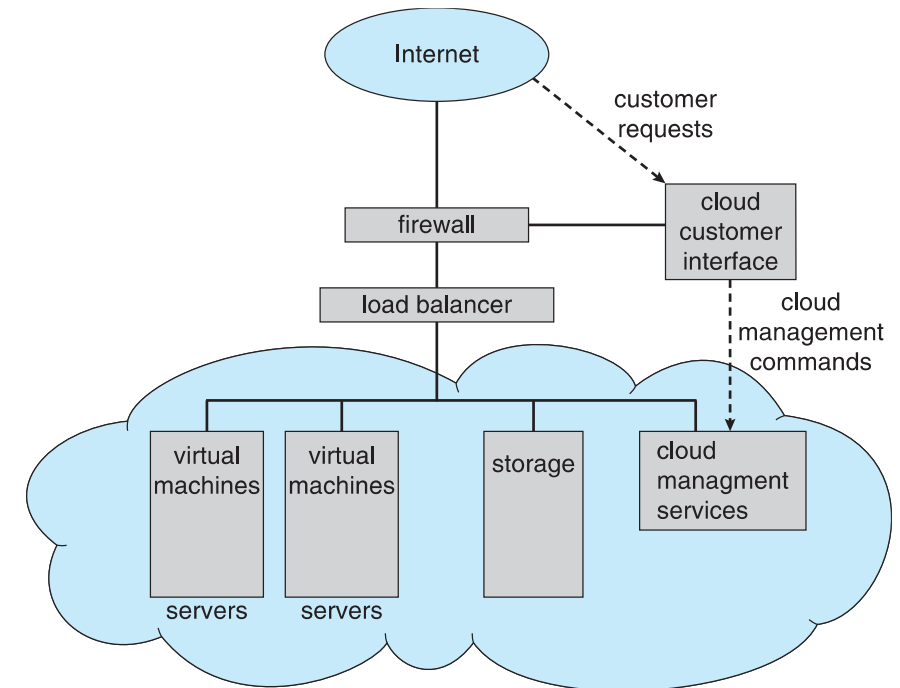
Many types

- **Public cloud** – available via Internet to anyone willing to pay
- **Private cloud** – run by a company for the company's own use
- **Hybrid cloud** – includes both public and private cloud components
- Software as a Service (**SaaS**) – one or more applications available via the Internet (i.e., word processor)
- Platform as a Service (**PaaS**) – software stack ready for application use via the Internet (i.e., a database server)
- Infrastructure as a Service (**IaaS**) – servers or storage available over Internet (i.e., storage available for backup use)

Computing Environments – Cloud Computing

Cloud computing environments composed of traditional OSeS, plus VMMs, plus cloud management tools

- Internet connectivity requires security like firewalls
- Load balancers spread traffic across multiple applications



Computing Environments – Real-Time Embedded Systems (/RTOS)

Real-time embedded systems most prevalent form of computers

- Vary considerable, special purpose, limited purpose OS, **real-time OS**
- Use expanding

Many other special computing environments as well

- Some have OSES, some perform tasks without an OS

Real-time OS has well-defined fixed time constraints

- Processing ***must*** be done within constraint
- Correct operation only if constraints met

Open-Source Operating Systems

Operating systems made available in source-code format rather than just binary **closed-source**

Counter to the **copy protection** and **Digital Rights Management (DRM)** movement

Started by **Free Software Foundation (FSF)**, which has “copyleft” **GNU Public License (GPL)**

Examples include **GNU/Linux** and **BSD UNIX** (including core of **Mac OS X**), and many more